NovusPal technical documentation

NovusPal is a local retrieval-augmented chat for public Michigan Technological University pages. FastAPI serves a single-page chat UI and a small HTTP API. Answers are generated on the machine that runs the server. Campus text is not sent to Gemini, OpenAI, or any other hosted model API.

Version: API `0.3.0` (`app/main.py`).

Scope

The index holds public pages only. Banweb, Canvas, and any other login-gated student records are out of scope. That boundary is a FERPA constraint, not only a scraper limitation.

The chatbot answers from chunks already stored in the vector database. It does not browse the live web at question time, and it does not keep a server-side transcript. The browser sends the last few turns with each question.

Architecture

```
Browser (app/static/index.html)
        |  POST /chat  (SSE)
        v
FastAPI (app/main.py)
        |
        +-- app/rag.py
        |     retrieve -> pack notes -> LangChain prompt -> local model
        |
        +-- app/store.py  ---- app/embed.py (sentence-transformers)
        |     chroma (default) or pgvector
        |
        +-- app/ingest.py + app/scraper.py
              poll public sources, hash, chunk, upsert, prune
```

Module	Role
`app/main.py`	HTTP API, static page, lifespan (warmup + scheduler)
`app/rag.py`	Retrieval, prompt, streaming generation
`app/embed.py`	Local embeddings and character chunking
`app/store.py`	Chooses Chroma or pgvector
`app/store_chroma.py`	Default persistent vector store
`app/store_pg.py`	Optional Postgres/pgvector store
`app/scraper.py`	robots.txt, delay, extract, feed/page/crawl
`app/ingest.py`	Change detection, prune, APScheduler
`app/config.py`	Settings and the source list
`app/static/index.html`	Chat UI

LangChain is used only to assemble the prompt and run the model (`ChatPromptTemplate` piped to a runnable and `StrOutputParser`). Scraping, chunking, and the vector store stay in this project. `LANGCHAIN_TRACING_V2` and `LANGSMITH_TRACING` default to `false` so page text is not sent to LangSmith.

Memory decision

A vector store is the memory. A question should retrieve chunks that mean the same thing, including paraphrases. A keyword document store would miss those. A graph database is not used because the app does not follow relationships between records (for example, major to advisor).

Chroma and pgvector store vectors. They do not create them. `app/embed.py` embeds both stored chunks and the question with the same local model.

Chunks live inside the vector database. There is no separate document database. Each chunk's metadata includes `url`, `title`, `category`, `source`, `content_hash`, `chunk`, and `ingested_at`.

Embeddings and chunking

Default model: `sentence-transformers/all-MiniLM-L6-v2`.

`embed()` loads that model once under a lock, encodes with `normalize_embeddings=True`, and returns plain lists of floats. Do not switch this path to LangChain `HuggingFaceEmbeddings`. That implementation strips newlines, which would not match vectors already stored from this encoder.

`chunk()` splits on paragraphs, then packs paragraphs up to `CHUNK_CHARS` (default 1200) with `CHUNK_OVERLAP` (default 150). A paragraph longer than the limit is sliced with the same overlap.

Search score is cosine similarity: Chroma returns distance and the store reports `1 - distance`; pgvector uses `1 - (embedding <=> query)`.

Ingestion

Sources are declared in `app/config.py`:

Name	Category	Kind	Start URL	`refresh_minutes`
`mtu-news`	`news`	`feed`	`https://www.mtu.edu/news/`	15
`mtu-events`	`events`	`page`	`https://www.mtu.edu/events/`	30
`library`	`library`	`crawl`	`https://www.mtu.edu/library/`	360
`tutoring`	`tutoring`	`crawl`	`https://www.mtu.edu/learning-center/`	360
`registration`	`registration`	`crawl`	`https://www.mtu.edu/registrar/`	720

Kinds:

- `feed` — RSS, Atom, or a landing page. If the response is XML, up to 50 entry links are fetched as pages. If it is HTML, the scraper follows a linked RSS/Atom `link` tag, or crawls links under that page's path.
- `page` — fetch the listed URLs only.
- `crawl` — breadth-first from the start URLs, same host, paths matching `link_patterns`, skipping `pdf`, `jpg`, `png`, `zip`, `doc`, `docx`, `xls`, and `xlsx`. Stops after `MAX_PAGES_PER_SOURCE` (default 60) URLs seen.

Fetching:

- Identifies itself with `USER_AGENT`.
- Reads `robots.txt` per host and skips disallowed URLs.
- Waits `REQUEST_DELAY_S` (default 1 second) between requests to the same host.
- Times out a plain fetch at 20 seconds. Playwright renders with a 30 second `networkidle` timeout when `render_js=True`.
- Extracts main text with Trafilatura (`include_tables=True`). If that text is shorter than 200 characters, it falls back to visible text after dropping `script`, `style`, `nav`, `footer`, and `header`.
- Ignores non-HTML responses for page extraction.

`render_js` defaults to false. Turning it on requires `pip install -r requirements-optional.txt` and `playwright install chromium`. If Playwright is missing, that source falls back to a plain fetch.

Change detection uses SHA-256 of the extracted text. An unchanged hash skips re-embedding. A changed page deletes its old chunks, then inserts new ones. After a source returns at least one document, URLs of that source that were not in the latest fetch are pruned. A fetch that returns nothing does not prune, so a temporary failure does not wipe the index.

One ingest runs at a time (`ingest._lock`). Startup launches one daemon thread per source; those threads queue on the lock. The scheduler is `BackgroundScheduler` in timezone `America/Detroit`. Each source is also registered as an interval job with `next_run_time=None`. APScheduler treats that as a paused job, so the interval does not fire by itself. A refresh after startup is a manual `POST /ingest` or `POST /ingest/{source_name}`.

`GET /health` includes `last_runs`, one object per source that has finished in this process: `pages`, `new`, `updated`, `unchanged`, and `pruned`.

`https://www.mtu.edu/events/` and `https://www.mtu.edu/learning-center/` have returned HTTP 404. Those sources then contribute no pages until the URLs in `SOURCES` are updated.

Answering a question

`POST /chat` calls `rag.stream_chat`.

1. Emit `{ "type": "status" }`.
2. Embed the question and search the store with `top_k` (default 4, max 8). If a category filter returns nothing, search again with no category.
3. Pack hits by URL. Text from the same URL is merged up to 900 characters. The higher score is kept.
4. If nothing was retrieved, stream a short “pages not loaded” sentence and `done`. The model is not called.
5. Build a prompt: system instructions, up to six prior turns (each clipped to 700 characters), packed notes, and the question.
6. Stream tokens from the local model. One generation holds `_gen_lock`, so a second question waits.
7. Emit `sources` (title and URL for notes that have a URL) and `done`.

The system prompt tells the model to answer in the first sentence, copy clock times, dates, and numbers exactly, and omit citation markers and URLs. The UI shows sources separately and strips `[n]` markers if the model still writes them.

Empty model output becomes a “could not find a clear answer” sentence, then sources and `done`. If tokens were already sent and generation then raises, the stream still ends with sources and `done` instead of an error. If nothing was sent, the client receives an `error` event. Ollama connection failures use a message that names Ollama. Other download or connection failures mention the network.

Generation settings for the Transformers backend: `max_new_tokens` from settings (default 320), `temperature` 0.4, `top_p` 0.9, `repetition_penalty` 1.08, sampling on. Weights load as `torch.float32` on CPU. The first load downloads `LLM_MODEL` into the Hugging Face cache.

`LLM_BACKEND=ollama` uses `langchain_ollama.ChatOllama` at `OLLAMA_BASE_URL` with the same temperature and `num_predict`. Any other backend raises at chain build time.

Chat history is not written to disk. The browser keeps successful turns (assistant text clipped to 1500 characters) and sends the last six. The API accepts at most eight turns, each at most 2000 characters, role `user` or `assistant`.

HTTP API

Interactive schema: `http://127.0.0.1:8000/docs` while the server is running.

Method	Path	Auth	Behavior
`GET`	`/`	none	`app/static/index.html`, `Cache-Control: no-cache`
`GET`	`/health`	none	`{ status, backend, chunks, llm_backend, llm_model, last_runs }`
`GET`	`/sources`	none	Configured sources: name, category, kind, urls, refresh_minutes
`GET`	`/search`	none	Raw hits. Query `q` (required), `k` (default 5), `category` (optional)
`POST`	`/chat`	none	SSE stream. Body is `ChatIn`
`POST`	`/ingest`	`X-API-Key`	Run every source. Returns a map of source name to ingest result
`POST`	`/ingest/{source_name}`	`X-API-Key`	Run one source. Unknown name is HTTP 404

`ChatIn`:

```
{
  "message": "What are the library hours?",
  "category": null,
  "top_k": 4,
  "history": [{ "role": "user", "content": "..." }, { "role": "assistant", "content":
"..." }]
}
```

`category` is a free string filter on chunk metadata. The UI sends `news`, `events`, `library`, `tutoring`, `registration`, or `null`.

SSE events are `data: {json}\n\n`:

type	Fields	Meaning
`status`	none	Retrieval has started
`token`	`text`	Next piece of the answer
`sources`	`sources`: `{ n, title, url }[]`	Pages used for the notes
`done`	none	Stream finished
`error`	`text`	Model failed before any tokens, or the request handler caught an exception

Response headers: `Content-Type: text/event-stream`, `Cache-Control: no-cache`, `X-Accel-Buffering: no`.

Admin routes compare `X-API-Key` to `ADMIN_API_KEY`. A mismatch is HTTP 401. The code default is `novuspal`; `.env.example` sets `change-me`. Set a long random value before anyone else can reach the port.

CORS middleware is installed only when `CORS_ORIGINS` is non-empty. It allows `GET` and `POST` from those origins. Empty means the browser UI is same-origin only.

Frontend

`app/static/index.html` is the whole UI. There is no separate JavaScript build.

- Topic buttons set `category` for the next request. `All` sends `null`.
- Starter cards submit a fixed question.
- Enter sends. Shift+Enter inserts a newline. The textarea grows up to 140px and rejects more than 2000 characters.
- The client parses SSE blocks split on `\n\n`, concatenates `token` text, and renders a small Markdown subset: paragraphs, `*` and `-` lists, numbered lists, `` `code` ``, `**bold**`, and `*italic*`.
- Source chips are links with `target="_blank"` and `rel="noopener"`.
- Colors follow `prefers-color-scheme`. There is no in-page theme switch.
- The send button stays disabled until the stream ends. A failed request shows “Something went wrong. Please try again.”

Configuration

Pydantic Settings reads `.env` (`extra="ignore"`). See `.env.example`.

Variable	Default	Effect
`LLM_BACKEND`	`transformers`	`transformers` or `ollama`
`LLM_MODEL`	`Qwen/Qwen2.5-0.5B-Instruct`	Hugging Face id, or an Ollama tag when the backend is `ollama`
`OLLAMA_BASE_URL`	`http://127.0.0.1:11434`	Ollama server
`EMBED_MODEL`	`sentence-transformers/all-MiniLM-L6-v2`	Local embedding model
`CHROMA_DIR`	`./chroma_data`	Chroma persistent path
`USER_AGENT`	`NovusPalBot/0.1`	Scraper identity. The example adds a contact email
`ADMIN_API_KEY`	`novuspal` in code	`X-API-Key` for ingest
`REQUEST_DELAY_S`	`1.0`	Delay between hits to one host
`MAX_PAGES_PER_SOURCE`	`60`	Crawl cap
`CHUNK_CHARS`	`1200`	Chunk size
`CHUNK_OVERLAP`	`150`	Overlap for long paragraphs
`VECTOR_BACKEND`	`chroma`	`chroma` or `pgvector`
`DATABASE_URL`	`postgresql://postgres:postgres@localhost:5432/novuspal`	Used when backend is `pgvector`
`CORS_ORIGINS`	empty	Comma-separated browser origins
`GEMINI_API_KEY`	empty	Ignored. Chat does not call Gemini

`max_new_tokens` is a settings field (default 320) and is not listed in `.env.example`. Set `MAX_NEW_TOKENS` if you need a different length.

Changing `EMBED_MODEL` or chunk sizes does not rewrite existing vectors. Delete `chroma_data` (or the Postgres `chunks` table) and ingest again so every vector comes from the same encoder.

Run and deploy

Local server, from the project directory:

```
.\.venv\Scripts\python.exe -m uvicorn app.main:app --host 127.0.0.1 --port 8000
```

There is no `--reload`. Code changes need a process restart. Use one worker. The scheduler and the in-process model lock assume a single process. For more than one web process, run ingestion outside the web workers.

Dependencies are in `requirements.txt`. Optional Postgres and Playwright packages are in `requirements-optional.txt`.

Chroma needs no extra service. `chroma_data/` is local state and is gitignored.

pgvector:

```
docker compose up -d db
```

Then install the optional requirements, set `VECTOR_BACKEND=pgvector`, and restart. Startup creates the `vector` extension, the `chunks` table, URL and source indexes, and an HNSW cosine index. Switching backends does not copy chunks. Call `POST /ingest` to fill the new store. `docker-compose.yml` uses password `postgres` and database `novuspal` on port 5432. That is a local default, not a production secret.

Lifespan starts two daemon threads before accepting traffic: one embeds the probe string `library hours`, the other loads the Transformers weights (or builds the Ollama chain). The server can answer `/health` before either finishes. `/chat` waits on the model lock when the weights are still loading.

Security

- Do not commit `.env`. It is gitignored. `.env.example` has no model API key.
- Treat `ADMIN_API_KEY` as a secret. Ingest can spend a long time fetching and embedding.
- `/search` and `/health` are unauthenticated and return stored page text and counts. Bind the server to localhost, or put it behind an authenticated proxy, before exposing it beyond one machine.
- Add rate limiting before a public `/chat`. The process generates one answer at a time on CPU.
- The scraper must stay on public URLs. Do not point `SOURCES` at authenticated systems.

Operational limits

- The default 0.5B model runs on CPU in float32. The first question after a cold start includes model load and can take minutes. Later questions are faster but still much slower than a hosted API.
- A small local model can blur schedules even when the retrieved notes are right. The prompt requires exact times and numbers; it cannot guarantee them.
- Interval jobs are registered paused. Freshness after the startup fetch depends on `POST /ingest`.

- Events and tutoring start URLs may 404, so those categories can be empty.
- `GET /search` embeds the query on the request thread. A cold embedding model blocks that request.
- Hugging Face on Windows may warn that it cannot create cache symlinks. The cache still works without Developer Mode.